

National University of Computer and Emerging Sciences



Laboratory Manual

for

Data Structures Lab

Course Instructor	Mr. Muhammad Naveed
Lab Instructor	Mr. Durraiz Waseem
Semester	Fall 2025

Department of Computer Science

FAST-NU, Lahore, Pakistan

Objectives:

In this lab, students will practice BST

Problem 1: Binary Tree and BST Implementation:

Implement the following Tree Node:

```
struct Node
{
    int data;
    Node* left;
    Node *right;
};
```

```
class BST
{
    Node* root;
};
```

Implement the following member functions for BST:

1. A default **Constructor** that sets the root to nullptr.
2. Implement a function '**insert**'. It should insert the data while considering the insertion rules. If the data already exists in the BST, simply return false and true otherwise. `bool insert(int v)`
3. A **copy constructor** which uses recursion to deep copy another Binary Search Tree object.
4. **Print** the tree on the screen
5. A function "inorderPrint" prints the keys using in-order traversal. `void inorderPrint () const`
6. Use level order traversal for the printing of trees, level by level. `void levelorderPrint () const`
7. A function "search". The function then uses recursion to return a pointer to the corresponding node. If the key does not exist, the function returns nullptr. `Node* search(int key)`
8. Use **Postorder** LRV to implement the Destructor for BST.
9. **BST to Doubly Linked List**: Convert a BST to a doubly linked list (DLL) in-place. The left and right pointers of nodes in the BST should act as the prev and next pointers of the DLL, respectively. **Hint**: Use in-order traversal and keep track of the previous node.
10. **Count Pairs from Two BSTs that Add Up to a Target**: Given two BSTs, find the number of pairs (one from each tree) where the sum of the pair equals a given target. **Hint**: Traverse one BST and for each value, search for the complement in the other BST efficiently.
11. **Kth Largest Element in a BST (Without Extra Memory)**: Find the K-th largest element in a BST without using any additional memory (no arrays or lists allowed). **Hint**: Use reverse in-order traversal and keep a counter until the k-th largest element is reached.